Propriedades

display

Block-level element

- Expande-se naturalmente para ocupar o seu *container*
  - A largura do elemento pode ser controlada (*width*)
  - Provoca uma quebra de linha
- Permite a definição de *margins* e/ou *padding*
- Se não for especificada a altura *(height)* o elemento expande-se para englobar os seus elementos filho
  - Este comportamento é alterado se o posicionamento dos filhos tiver sido alterado (*float/position*)
- A sua posição original é definida pelo fluxo do HTML
- Exemplos:
  - <p>, <h1>, , <form>, <ul>, <li>, …



Block-level element

- *Exemplo: expansão do elemento*

```
        div{ background-color:rgb(245, 245, 245);
        color:darkgray;
        padding-left:10px;
        border-top:1px lightgray solid;
        border-bottom:1px lightgray solid;
        padding-top: 15px;
        padding-bottom: 15px;}
    </style>
</head>
<body>

    <div id="div1">div1</div>
    <div id="div2">div2</div>
    <div id="div3">div3</div>
```

** ** é um ***block level element***

Caso não seja definida a largura, o elemento expande-se considerando a totalidade da largura do *viewport*



Block-level element

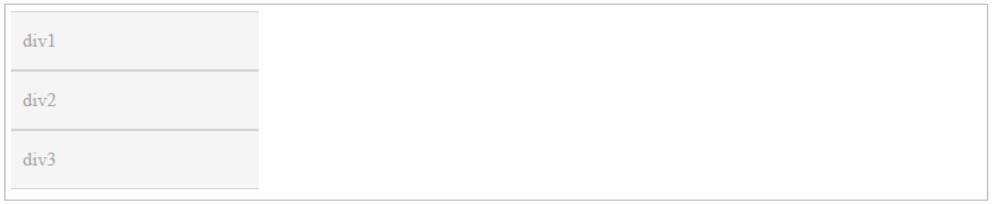
- *Exemplo: propriedade width*

```
        div{ background-color:rgb(245, 245, 245);
        color:darkgray;
        padding-left:10px;
        border-top:1px lightgray solid;
        border-bottom:1px lightgray solid;
        padding-top: 15px;
        padding-bottom: 15px;
        width:200px}
    </style>
</head>
<body>

    <div id="div1">div1</div>
    <div id="div2">div2</div>
    <div id="div3">div3</div>
```

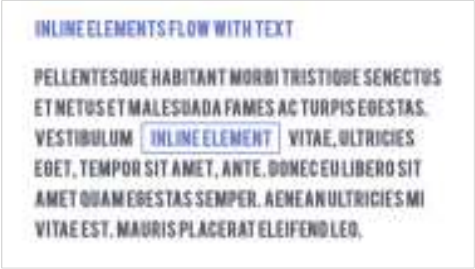
** ** é um ***block level element***

Caso a largura seja definida, o elemento não se expande assumindo a dimensão estabelecida.



Inline-level Element

- Aplica-se a conteúdo textual
  - Pode ser percepcionado como uma caixa que se comporta como texto.
  - **Não se expande** à totalidade do espaço disponível (container/viewport)
- **Ignora** margens topo/fundo (*top/bottom*) mas permite a definição de *left/right margin* assim como a propriedade *padding*.
- **Ignora** as propriedades largura e altura (width/height).
  - Se for definido como um elemento *float (left/right)* torna-se automaticamente um *block-level element*
- Está sujeito a alinhamento vertical (*vertical-align*)
- Exemplos:
  - <a>, , …



*{display:***}*

- *display*
  - propriedade muito importante nomeadamente na conceção de menus de navegação

Propriedade	*Exemplo*	*Observações*
display	li {display:inline;}	Permite definir a disposição de elementos. Valores: none; inline; block; inline-block; flex; …

{display:***}

- seletor{*display:**inline***}
  - Define o elemento como um ***inline element***
  - Muito útil para a implementação de menus horizontais *(horizontal navigation bar)*

```
      div{ background-color:rgb(245, 245, 245);
        color:darkgray;
        padding-left:10px;
        border-top:1px lightgray solid;
        border-bottom:1px lightgray solid;
        padding-top: 15px;
        padding-bottom: 15px;
        width:200px;
        }
     div{display:inline}
    </style>
</head>
<body>

    <div id="div1">div1</div>
    <div id="div2">div2</div>
    <div id="div3">div3</div>
```



{display:***}

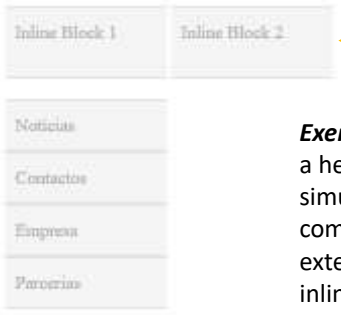
- seletor{display: **block**}
  - Define o elemento como um ***block element***
  - Muito útil para a implementação de menus verticais *(vertical navigation bar)*

```
    <style>
        a{
            font:Verdana;
            font-size: 1.1em;
            display:block;
            height:20px;
            width:150px;
            text-decoration: none;
            color:darkgray;
            padding-left:10px;
            border-top:1px lightgray solid;
            border-bottom:1px lightgray solid;
            border-radius: 1px;
            padding-top: 15px;
            padding-bottom: 15px;
            background-color: rgb(245, 245, 245);
        }
    </style>
    <title>Document</title>
</head>
<body>
    <a href="">Noticias</a>
    <a href="">Contactos</a>
    <a href="">Empresa</a>
    <a href="">Parcerias</a>
</body>
</html>
```



{display:***}

- seletor{display:**inline-block**}
  - O interior do elemento é considerado como um *block-level element*
  - O elemento é definido como um *inline-level element*
  - Mais versátil que *display:inline*



Exemplo: alterar a height e em simultâneo ter o comportamento exterior de um inline element

```
        a{  font:Verdana;
            font-size: 1.1em;
            display:block;
            height:20px;
            width:150px;
            text-decoration: none;
            color:darkgray;
            padding-left:10px;
            border-top:1px lightgray solid;
            border-bottom:1px lightgray solid;
            border-radius: 1px;
            padding-top: 15px;
            padding-bottom: 15px;
            background-color: rgb(245, 245, 245);}
        a.c1{display:inline-block;
             height:40px;
             margin-bottom: 20px}
    </style>
</head>
<body>
    <a href="" class="c1">Inline Block 1</a>
    <a href="" class="c1">Inline Block 2</a>

    <a href="">Noticias</a>
    <a href="">Contactos</a>
```


{display:***}

- seletor{display:**none**}
  - O elemento não é visualizado **nem tem qualquer** influência no layout

```
        a.c1{display:inline-block;
             height:40px;
             margin-bottom: 20px}
        a.c2{display:none}
    </style>

</head>
<body>
    <a href="" class="c1 c2">Inline Block 1</a>
    <a href="" class="c1">Inline Block 2</a>
```



  - lista completa dos valores propriedade ***display***
    - http://www.w3schools.com/cssref/pr_class_display.asp

Layouts

Tipos

Layouts

- Existem diferentes tipos de *layouts*:
  - ***fluid layout/flexible layout***
    - As dimensões são estabelecidas de forma a existir um redimensionamento dos conteúdos quando as dimensões do *viewport* são alteradas
  - *fixed layout*
    - As dimensões são especificadas em ***px*** sem atender às dimensões do *viewport* ou do tamanho do texto

fluid layout/flexible layout

- As colunas contidas na página ajustam-se ao espaço disponível na janela do browser
  - layout versátil que se ajusta a diferentes dimensões da janela do browser, e como tal pode ser um elemento chave do *responsive web design* (adaptar conteúdos a diferentes *viewports*)
  - As dimensões são definidas em % ou tiram partido de novas abordagens CSS
    - Tipo de layout que atualmente deve ser implementado!

Vantagens	Desvantagens
Ajustam o conteúdo ao espaço de visualização disponível;	
Permite a correta visualização em maior variedade de dispositivos (*desktops, mobile, …*);	É menos previsível que os layouts fixos, nomeadamente para dimensões extremas dos browsers;
Evita margens muito grandes (espaço vazio);	
Evita a necessidade de recorrer a *scrollbars* horizontais.	

fixed layout

- Largura fixa em pixéis (px)
  - Layout tradicional uma vez que a esmagadora maioria dos acessos era efetuada a partir de *desktops*
    - **Esta situação já não se verifica !!**
      - ex: 1024 x 768 (width: 960px)
  - Mesmo para uma dimensão fixa, inflexível e dependente do alinhamento relativamente à janela do browser:
    - à esquerda
      - margem **variável** do lado direito (resulta num layout muito desequilibrado);
    - centrado
      - garantido margens (esquerda/direita) de largura igual, o que melhora o enquadramento do conteúdo

Layouts

Tipos

Abordagens

Layout

- CSS FlexBox
  - Baseia-se num conjunto de propriedades que permitem a disposição flexivel dos elementos ao longo de um eixo.
- CSS Grid Layout
  - Baseia-se na criação de um grid container para disposição dos elementos por linha/coluna
  - Suportado pelos browsers (sem vendor prefix) a partir de 2017
- Frameworks CSS
  - Forma rápida e poderosa de criação de um layout (exemplo:Bootstrap)
  - Aprendizagem de um novo framework
- *float /clear*
  - Implementação simples. Ligado ao fluxo do HTML o que reduz a sua flexibilidade

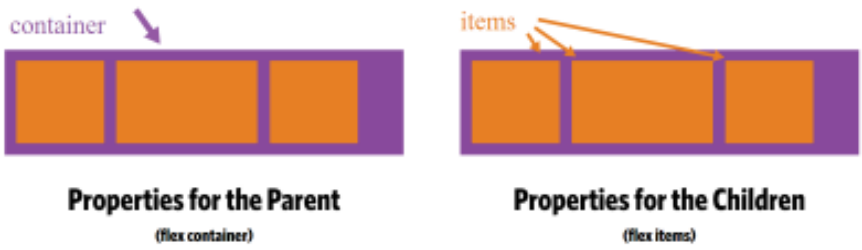
CSS Flexbox (Layouts)

CSS Flexbox

- *flexbox container*
  - permite agilizar a definição de layouts

*According to the specification, the Flexbox model provides for an efficient way to **layout, align, and distribute space among elements within your document** — even when the viewport and the size of your elements is dynamic or unknown.*

https://medium.freecodecamp.org/understanding-flexbox-everything-you-need-to-know-b4013d4dc9af



- *{display:flex}*

https://css-tricks.com/snippets/css/a-guide-to-flexbox/

  - cria um *container* flexível com um conjunto de propriedades para o *container* e para os respetivos *items*

- Pode ser aplicado a diversos elementos
  - ul -> flex container
  - li -> flex item (filhos do flex container)

```
<ul>
    <li>1</li>
    <li>2</li>
    <li>3</li>
</ul>
```

```
ul{display:flex;}

li {    width: 100px;
       height: 100px;
       background-color:lightblue;
       margin:8px;

       list-style-type:none;
       text-align:center;
       color:white}
```

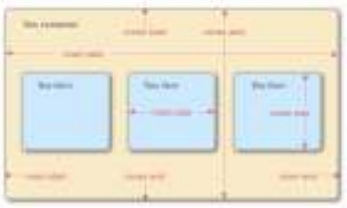

CSS Flexbox (Layouts)

flex container

flex item

CSS Flexbox *(layout)*

- Propriedades do ***flex container***:



  - *flex-direction*
    - Estabelece a direção do eixo principal ao longo do qual os elementos são dispostos
  - *flex-wrap*
    - Define se o *container* adapta a sua dimensão ou se cria múltiplas linhas para conter os *flex items*
    - ***flex-flow***
      - propriedade agregada de *flex-direction* e *flex-wrap* (ex: *flex-flow: row wrap*;)
  - align-items
    - define o alinhamento dos *items* ao longo do *cross axis*
  - align-content
    - define o alinhamento dos *items* ao longo do *cross axis* quando existe mais do que uma linha
  - justify-content
    - define o alinhamento dos *items* ao longo do *main axis*

CSS Flexbox *(layout)*

- *Propriedades dos **flex container***

  - *flex-direction*: row |row-reverse | column | column-reverse



  - *flex-wrap*: nowrap|wrap|wrap-reverse
    - O comportamento por defeito é nowrap, o flex container adapta-se aumentando de tamanho para conter todos os flex items
    - O valor wrap cria múltiplas linhas



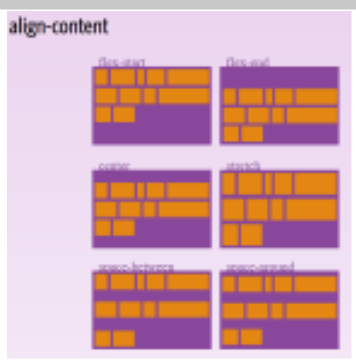
  - *align-items*: flex-start | flex-end | center | space-between | space-around | stretch;



https://css-tricks.com/snippets/css/a-guide-to-flexbox/

CSS Flexbox *(layout)*

- align-content: flex-start | flex-end | center | space-between |space-around | stretch;
  - **Não se aplica** quando existe apenas uma linha de *items*



- justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly;



https://css-tricks.com/snippets/css/a-guide-to-flexbox/

CSS Flexbox *(layout)*

Exemplo:

```
<div id="container">
    <div id="d1">Item1 Item1 Item1 Item1 Item1 Item1 Item1 </div>
    <div id="d2">Item2 Item2 Item2 Item2 Item2 Item2 Item2 </div>
    <div id="d3">Item3 Item3 Item3 Item3 Item3 Item3 Item3 </div>
</div>
```

```
#container{
     height:400px;
     width:400px;
     display:flex;
     flex-direction: row;
     align-items:flex-start;}
```

Item1 Item1 Item1 Item1 Item1 Item1 Item1
Item2 Item2 Item2 Item2 Item2 Item2 Item2
Item3 Item3 Item3 Item3 Item3 Item3 Item3

Na definição do *container* flexível através ***(display:flex)*** estabeleceu-se que os conteúdos seriam apresentados por linha ***(flex-direction:row)*** e que o alinhamento dos *items* seria feito a partir do topo do *container* ***(align-items:flex-start)***

Os elementos aproveitam a largura disponível (400px), são dispostos com a mesma largura e pela ordem que surgem no HTML

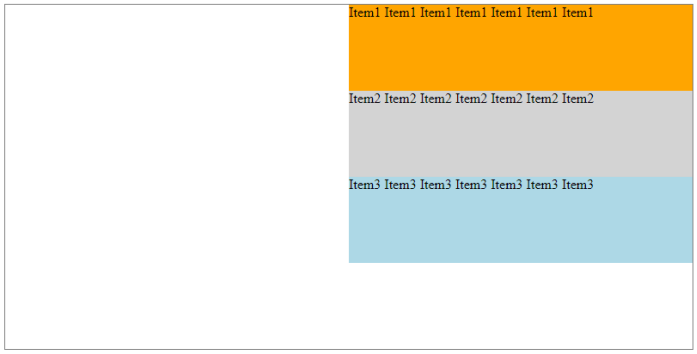
CSS Flexbox *(layout)*

```
<div id="container">
    <div id="d1">Item1 Item1 Item1 Item1 Item1 Item1 Item1 </div>
    <div id="d2">Item2 Item2 Item2 Item2 Item2 Item2 Item2 </div>
    <div id="d3">Item3 Item3 Item3 Item3 Item3 Item3 Item3 </div>
</div>
```

```
#d1{background-color: orange;}
#d2{background-color: lightgray;}
#d3{background-color: lightblue;}

#container{
    height:400px;
    width:800px;
    border: 1px gray solid;
    display:flex;
    flex-direction: column;
    align-items:flex-end;
}

#container div{
    height:100px;
    margin:0px;
    padding:0px;
    width: 400px;}
```

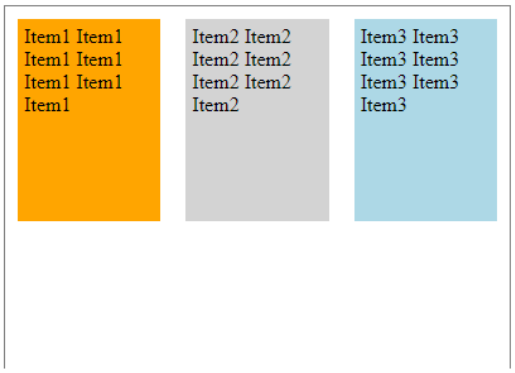


Tendo por base os mesmos elementos estruturais, é possivel alterar de forma radical a forma como são apresentados, controlando apenas as propriedades do *flex-container*

CSS Flexbox *(layout)*

- O flex container permite que sejam aplicadas propriedades aos flex items:
  - Exemplo: neste caso podem ser definidos o *padding, a margin e a height* de cada flex item

```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
}
```

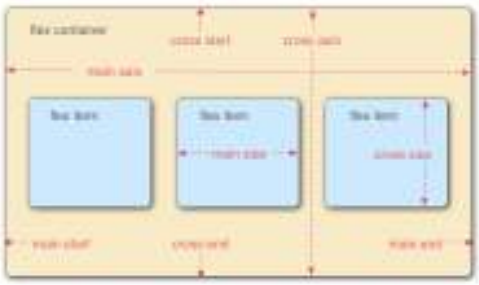


CSS Flexbox (Layouts)

flex container

flex item

CSS Flexbox (layout)



- *Propriedades dos **flex items***
  - *order*
    - permite reordenar os *flex items* num *container*
  - *flex-grow*
    - estabelece quanto é que um item pode crescer se existir espaço livre no *flex container*
  - *flex-shrink*
    - estabelece quanto é que um item pode reduzir caso não exista espaço no *flex container*
  - *flex-basis*
    - define a dimensão inicial de um *flex item*
  - *flex*
    - propriedade agregada (*flex: flex-grow flex-shrink flex-basis*)

CSS Flexbox *(layout)*

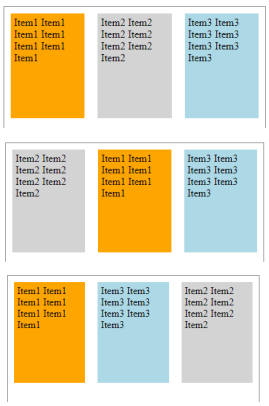
- Propriedades para os ***flex items***:
  - {*order: <integer>;*}
    - Por defeito a ordem dos *items* segue a ordem do HTML, sendo que todos os *flex items* possuem o valor 0 (*order; default value*).
      - É possível controlar a posição de um item controlando o valor desta propriedade
        - um valor negativo coloca o *item* em primeiro lugar
        - um valor positivo posiciona o item depois de todos aqueles com um valor de *order* inferior.

```
<div id="container">
    <div id="d1">Item1...</div>
    <div id="d2">Item2...</div>
    <div id="d3">Item3...</div>
</div>
```

```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
}
```

```
#d2{order:-1;}
```

```
#d2{order:1;}
```



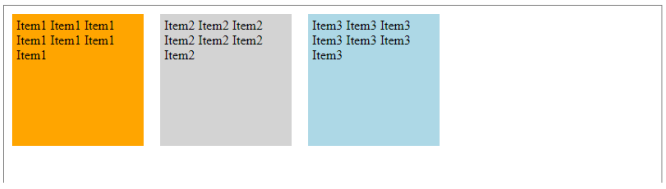
CSS Flexbox *(layout)*

- *{flex-grow: <integer>;}*
  - Estabelece a capacidade de um item para crescer
  - Aceita um valor que define a quantidade de **espaço disponível** que cada um dos *items* vai ocupar.
    - Se todos os *items* possuírem o valor 1 o espaço disponível é dividido de igual forma pelos diversos items
    - Caso contrário o item com o maior valor vai ocupar mais espaço do que os outros *items*.

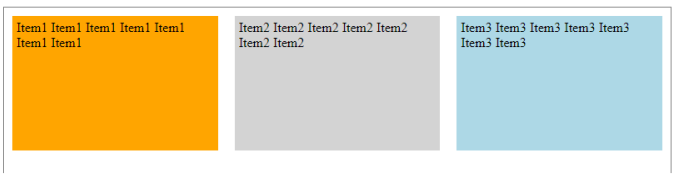
```
#container{
    height:400px;
    width:800px;
    border: 1px gray solid;
    display:flex;
    flex-direction: row;
    align-items:flex-start;}
```

```
#container div{ width:150px;
                height:150px;
                margin:10px;
                padding:5px;}
```

```
#container div{flex-grow: 1;}
```



Se largura/altura inicial dos diversos items é diferente, então as dimensões finais também serão diferentes apesar do espaço disponível ser distribuído de igual forma



CSS Flexbox *(layout)*

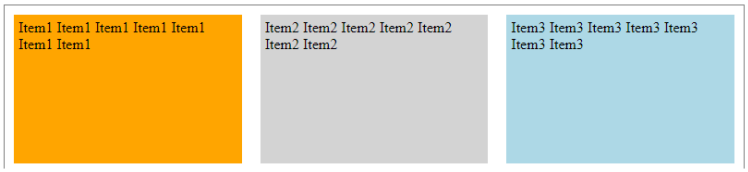
- {flex-shrink: <integer>;}
  - Capacidade de um item para reduzir a sua dimensão, o valor inteiro define o grau de redução atribuído a esse elemento.
  - Esta propriedade só se aplica em situação de **overflow** (largura items > largura do container)

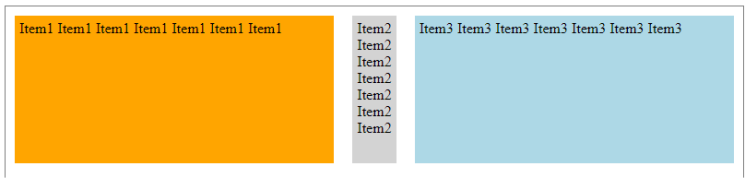
```
#container{
    height:400px;
    width:800px;
    border: 1px gray solid;
    display:flex;
    flex-direction: row;
    align-items:flex-start;}
```

```
#container div{
        height:150px;
        margin:10px;
        padding:5px;
        flex-grow: 1;
        flex-shrink: 1;
        width: 400px;}
```

```
#container #d2{flex-shrink:6;}
```

Como a largura está definida e existe uma situação de *overflow*, a dimensão dos *flex-items* irá ajustar-se à largura do *flex container*



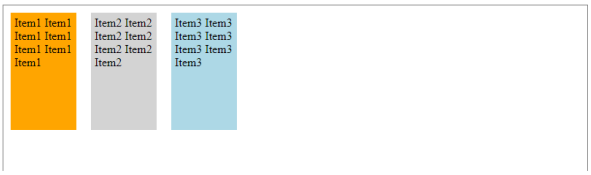


CSS Flexbox *(layout)*

- {flex-basis: <length> | auto;}
  - Define a dimensão (%, em, …) inicial de um flex item.

```
#container div{
          height:150px;
          margin:10px;
          padding:5px;
         /*flex-grow: 1;*/
         }

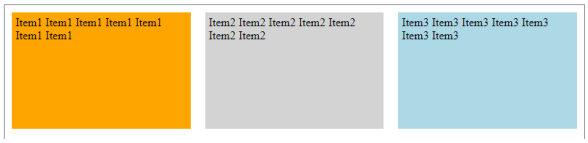
#container div{flex-basis:10%;}
```



- Se *flex-grow* definido o espaço livre é repartido pelos *flex-items*

```
#container div{
          height:150px;
          margin:10px;
          padding:5px;
          flex-grow: 1;
         }

#container div{flex-basis:10%;}
```

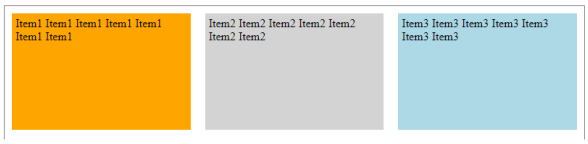


CSS Flexbox *(layout)*

- {flex-basis: <length> | auto;}
  - Se a largura do elemento não se encontrar definida, o valor **auto** ajusta ao conteúdo de cada um dos flex items
    - Caso a largura do *flex-item* tenha sido definida (*width*), *{flex-basis:auto}* não altera essa definição.

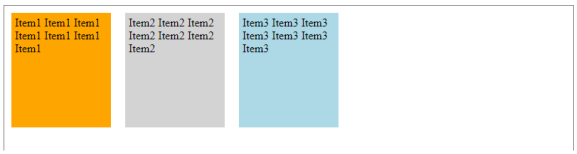
```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
 /*  flex-grow: 1; */
 /*   width: 130px;*/}

#container div{flex-basis:auto;}
```




```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
 /* flex-grow: 1; */
    width: 130px;}

#container div{flex-basis:auto;}
```

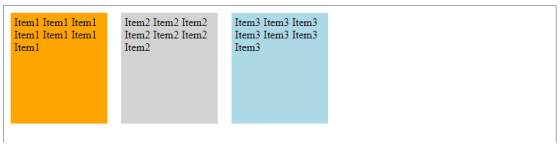


CSS Flexbox *(layout)*

- {flex: 0 1 auto}
  - propriedade agregada para os valores *flex-grow, flex-shrink* (opcional) e *flex-basis* (opcional) .
  - Os valores por defeito são (0 1 auto).

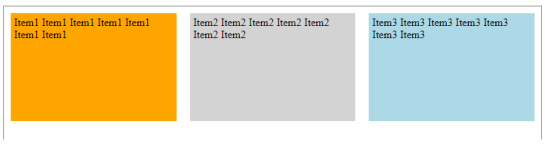
```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
    width: 130px;}

#container div{flex:0 1 auto;}
```




```
#container div{
    height:150px;
    margin:10px;
    padding:5px;
    width: 130px;}

#container div{flex:1 1 auto;}
```



- ***align-self***: auto || flex-start || flex-end || center || baseline || stretch
  - Permite o alinhamento de um *flex item*, mantendo o alinhamento previamente definido para os restantes *flex items*

```
#container div{
        height:150px;
        margin:10px;
        padding:5px;
        width: 130px;}

#container div{flex:1 1 auto;}

#d2{align-self:flex-end}
```



Exercício Seletores

Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element
<span>parágrafo1</span></span></p>

<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>

<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```


Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element
<span>parágrafo1</span></span></p>

<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>

<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element
<span>parágrafo1</span></span></p>

<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>

<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```


Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element
<span>parágrafo1</span></span></p>

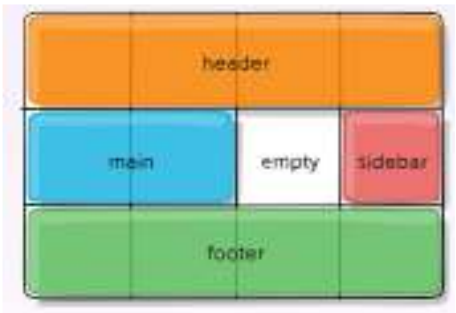
<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>

<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

CSS Grid Layout

CSS Grid Layout

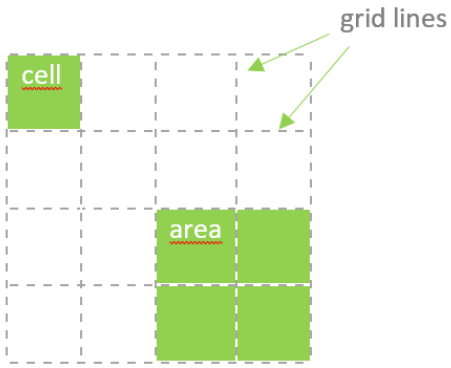
- A criação de um *grid container* é efetuada através de *{display: grid}*
  - Deve ser estabelecido o número de linhas/colunas do grid container
  - Todos os filhos **diretos** de um *grid container* tornam-se *grid items*
  - Associar cada *grid item* a uma área no *grid container*
    - pode ser criada uma grid dentro de uma outra grid e assim criar vários contextos de posicionamento



https://css-tricks.com/snippets/css/complete-guide-grid/

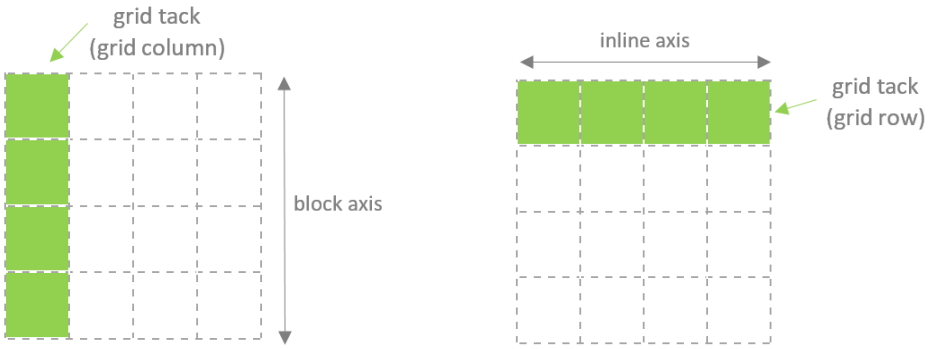
CSS Grid Layout

- CSS Grid
  - *line*
    - linhas horizontais e verticais que constituem a *grid*
  - *cell*
    - unidade mais pequena de uma *grid* a qual é delimitada por 4 *grid lines* adjacentes
  - *area*
    - área retangular constituida por uma ou mais *grid cells* adjacentes



CSS Grid Layout

- CSS Grid
  - *track*
    - o espaço entre células adjacentes é um *grid track*, o qual pode ser um *grid column* ou *grid row*
      - *grid column* é vertical (*block axis*)
      - *grid row* é horizontal (*inline axis*)



CSS Grid Layout

Grid Container

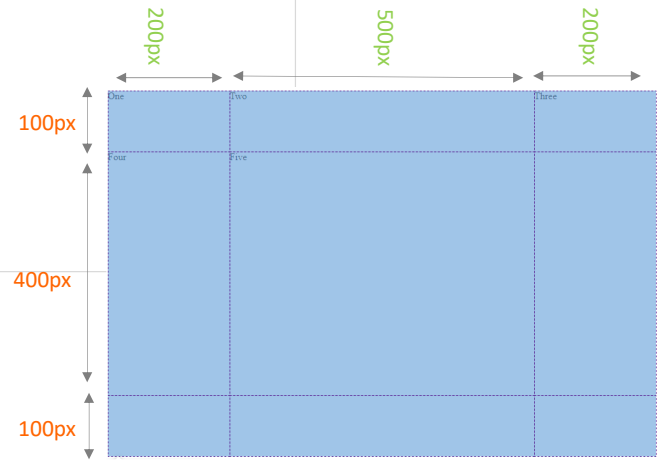
display: grid
grid-template-rows
grid-template-columns
grid-template-areas

Definição de um Grid Container

- A declaração de um CSS grid é feito através da propriedade **display:grid**
  - **grid-template-rows; grid-template-columns** definem n.º e dimensões de linhas/colunas

```
        #layout{display: grid;
                grid-template-rows: 100px 400px 100px;
                grid-template-columns: 200px 500px 200px}
    </style>
</head>
<body>

    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
        <div id="four">Four</div>
        <div id="five">Five</div>
    </div>
```



Definidas 3 linhas e 3 colunas.
O *template* pode ser dividido no número de linhas e colunas definidos pelos valores atribuidos a **grid-template-rows**; **grid-template-columns**

Definição de um Grid Container

- A propriedade **grid-template-areas** permite identificar as várias *grid cells*
  - Importante!
    - Trata-se apenas da identificação das células uma vez que a grid e as respetivas dimensões das grid tracks são definidas através das propriedades **grid-template-rows**, **grid-template-columns**

```
#layout{display: grid;
    grid-template-rows:  100px  400px 100px;
    grid-template-columns: 200px  500px 200px;
    grid-template-areas:
        "header header header"
        "ads main links"
        "footer footer footer"}
</style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
        <div id="four">Four</div>
        <div id="five">Five</div>
        <div id="six">Six</div>
    </div>
```

Células são referenciadas por **linha**

header	header	header
ads	main	links
footer	footer	footer

Definição de um Grid Container

- Existe ainda a propriedade **grid** que agrega as propriedades:
  - grid-template-rows
  - grid-template-columns
  - grid-template-areas
- não é muito utilizada uma vez que origina uma sintaxe mais complexa do que a utilização das propriedades individuais:

```
#layout{display: grid;
        grid:
                "header header header"  100px
                 "ads main links"       400px
                 "footer footer footer" 100px
                 / 200px  500px 200px
        }
```

Especificação feita por **linha** / dimensão das colunas

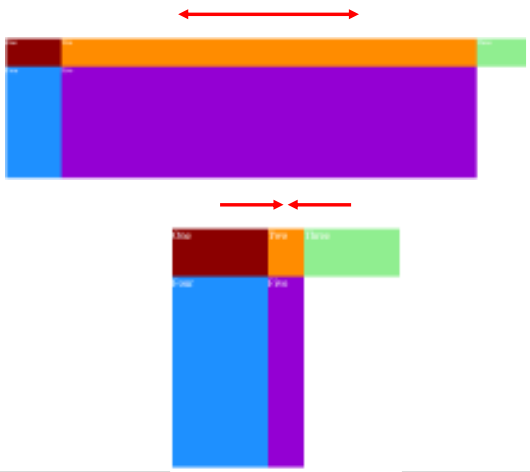
CSS Grid Layout

Grid Container

Unidades

Unidades

- Na CSS grid as unidades a utilizar devem assegurar a flexibilidade necessária para um *responsive web design* (diversidade de dispositivos)
  - px (fixed); %
  - fr *(fractional unit)*
    - permite que os grid tracks se ajustem ao espaço disponível, expandam/retraiam de acordo com as variações do *viewport*.



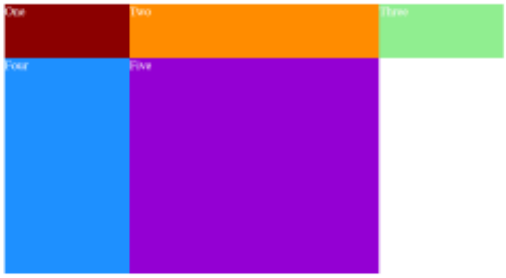
```
#layout{
    display: grid;
    grid-template-rows:  100px  400px 100px;
    grid-template-columns:  200px  1fr 200px}
```

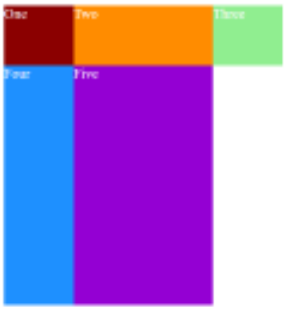
Unidades

- As *fractional units* podem ser usadas para:
  - combinar áreas de dimensão variável com dimensão fixa (exemplo anterior)
  - combinar áreas de dimensão variável

```
#layout{
    display: grid;
    grid-template-rows:  100px  400px 100px;
    grid-template-columns:  1fr  2fr 1fr}
```

A coluna central terá sempre o dobro da capacidade de ajustamento das colunas dos extremos





Unidades

- A função minmax() permite limitar as dimensões de uma grid track
  - tornar essa *grid track* flexível mas dentro de limites pré-definidos

```
#layout{
    display: grid;
    grid-template-rows:  100px  400px 100px;
    grid-template-columns:  200px  minmax(5em,15em) 200px}
```

  - As dimensões de uma grid track também podem ser definidas com base nas dimensões do seu próprio conteúdo
    - min-content
      - o menor valor para não criar situações de overflow (ex: a palavra mais longa; a imagem mais larga, …)
    - max-content
      - o valor máximo que permita conter o conteúdo sem wrapping.
        - **Atenção!** Não é adequado para parágrafos, o efeito pode ser imprevisivel.

Unidades

- As dimensões da grid podem ser definidas tirando partido de:
  - *repeat (number_of_repetitions, pattern)*

```
        #six{background-color:darksalmon}

        #layout{display: grid;
                grid-template-rows:  100px  400px 100px;
                grid-template-columns:  repeat(2, 200px  1fr 200px)}
    </style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
        <div id="four">Four</div>
        <div id="five">Five</div>
        <div id="six">Six</div>
    </div>
```



CSS Grid Layout

Grid Container

Unidades

Grid Items

Posicionamento Grid Items

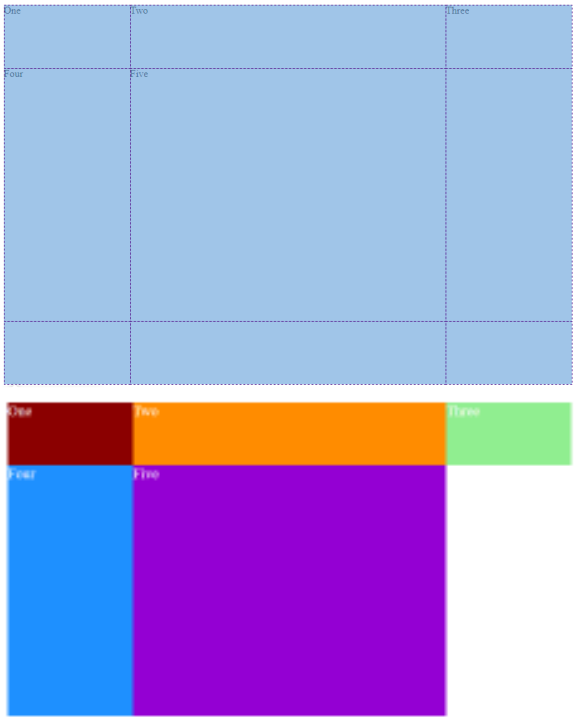
- Cada um dos *grid items* é atribuído de forma automática a cada uma das *grid cells*
  - por defeito essa atribuição é sequencial e feita por linha

```
        #layout{display: grid;
                grid-template-rows: 100px 400px 100px;
                grid-template-columns: 200px 500px 200px}

        div{color:white; font-size: 1.2em}
        #one{background-color:darkred}
        #two{background-color:darkorange}
        #three{background-color:lightgreen}
        #four{background-color:dodgerblue}
        #five{background-color:darkviolet}

    </style>
</head>
<body>

    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
        <div id="four">Four</div>
        <div id="five">Five</div>
    </div>
```



CSS Grid Layout

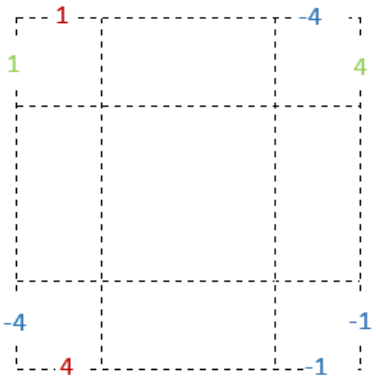
Grid Container

Unidades

Grid Items : Posicionamento

Grid Items: Posicionamento

- Identificação das linhas
  - Identificação numérica
    - row line (1 – 4)
    - column line (1-4)
    - Contagem Inversa
      - a contagem inicia-se na ultima linha sempre com o valor -1 e processa-se de forma descrescente (-2, -3, …)

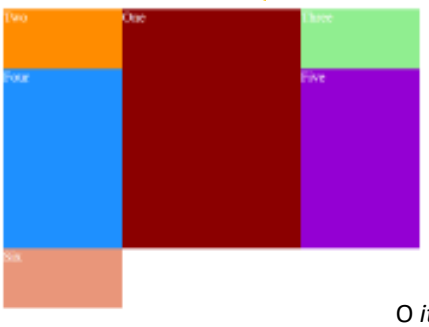


  - Identificação baseada em nomes […]

```
#layout{display: grid;
        grid-template-rows: [header-start] 100px  [content-start] 400px [footer-start] 100px;
        grid-template-columns: [adds] 200px [main] 500px  [links] 200px}
```


Grid Items: Posicionamento

- A alteração do posicionamento é efetuado a partir da identificação das linhas tendo por base as propriedades:
  - grid-row-start
  - grid-row-end
  - grid-column-start
  - grid-column-end

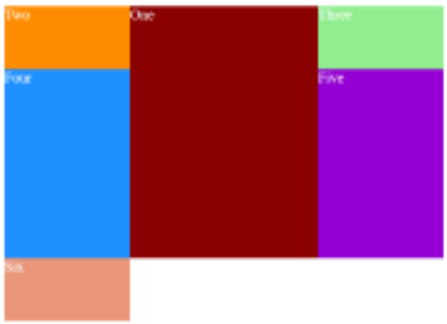


```
        #layout{display: grid;
                grid-template-rows:  100px  300px 100px;
                grid-template-columns: 200px  300px 200px;
                grid-template-areas:
                    "header header header"
                    "ads main links"
                    "footer footer footer"
                }
        #one{
            grid-row-start: 1;
            grid-row-end:3;
            grid-column-start: 2;
            grid-column-end: 3;
        }
    </style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
```

O *item* é posicionado, sendo que todos os outros reajustam de acordo com o espaço disponível
O posicionamento é feito pelo número/nome da linha

Grid Items: Posicionamento

- O posicionamento anterior pode ser efetuado de forma mais condensada:
  - grid-row: start line / end line
  - grid-column: start line / end line




```
            #layout{display: grid;
                grid-template-rows:  100px  300px 100px;
                grid-template-columns: 200px  300px 200px;
                grid-template-areas:
                    "header header header"
                    "ads main links"
                    "footer footer footer"
                }
        #one{
            grid-row: 1 / 3;
            grid-column: 2 / 3;

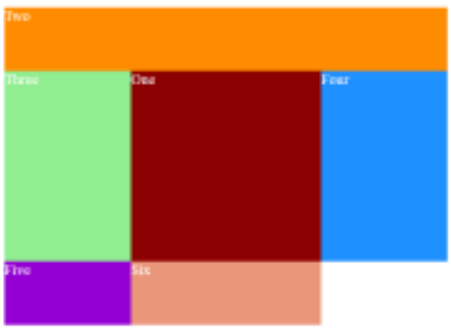
        }
    </style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
        <div id="four">Four</div>
        <div id="five">Five</div>
        <div id="six">Six</div>
    </div>
```


Grid Items: Posicionamento

- O posicionamento através de grid-area também pode ser feito com base na designação das áreas criadas através de **grid-template-areas**

```
        #layout{display: grid;
                grid-template-rows:  100px  300px 100px;
                grid-template-columns: 200px  300px 200px;
                grid-template-areas:
                    "header header header"
                    "ads main links"
                    "footer footer footer"
                }
        #one{
            grid-area: main;
        }
        #two{
            grid-area: header;
        }
    </style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
```

header	header	header
ads	main	links
footer	footer	footer

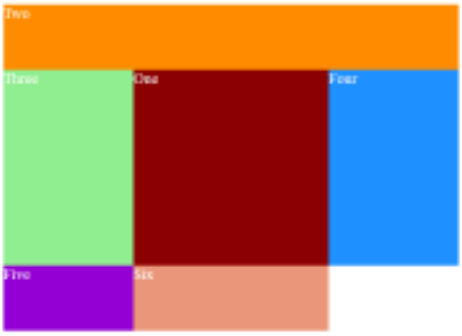


Grid Items: Posicionamento

- A propriedade **grid-area** pode também ser aplicada com base em 4 grid lines as quais definem uma área:
  - grid-area: *row-start / column start / row-end / column-end*;

```
        #one{
            grid-area: main;
        }
        #two{
            grid-area:1 / 1 / 2 / 4;
        }
    </style>
</head>
<body>
    <div id="layout">
        <div id="one">One</div>
        <div id="two">Two</div>
        <div id="three">Three</div>
```

header	header	header
ads	main	links
footer	footer	footer

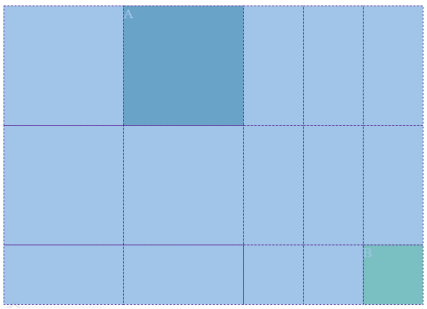


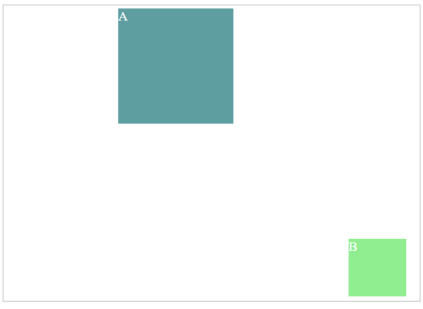
Grid Items: Posicionamento

- Se o posicionamento for definido **fora da grid inicial** podem ser geradas automaticamente linhas/colunas para esse posicionamento ser possível
  - ***grid-auto-rows*** e ***grid-auto-columns*** utilizadas para geração automática de *tracks* e aplicar à grid prédefinida

```
        div{color:white; font-size: 1.2em}
        #littlegrid{
            display: grid;
            grid-template-columns: 200px 200px;
            grid-template-rows: 200px 200px;
            grid-auto-columns: 100px;
            grid-auto-rows: 100px    }

        #a{
            grid-row: 1 / 2;
            grid-column: 2 / 3;
            background-color: cadetblue;}
        #b{
            grid-row: 3 / 4;
            grid-column: 5 / 6;
            background-color:lightgreen; }
    </style>
</head>
<body>
  <div id="littlegrid">
    <div id="a">A</div>
    <div id="b">B</div>
  </div>
```



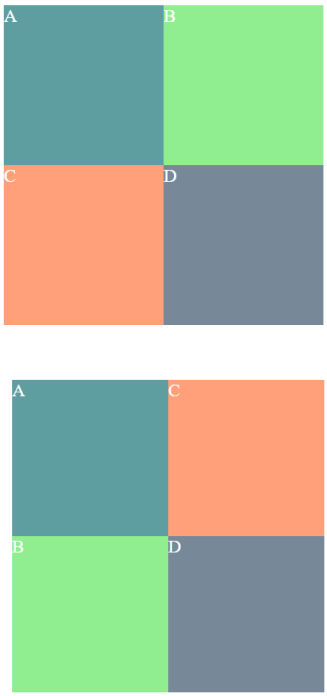


Grid Items: Posicionamento

- A propriedade **grid-auto-flow** permite estabelecer a direção do posicionamento (*row (default); column*)

```
            div{color:white; font-size: 1.2em}
            #littlegrid{
                display: grid;
                grid-auto-flow: column;
                grid-template-columns: 200px 200px;
                grid-template-rows: 200px 200px;
                grid-auto-columns: 100px;
                grid-auto-rows: 100px    }

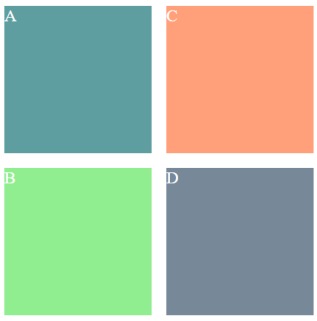
            #a{background-color: cadetblue;}
            #b{background-color:lightgreen; }
            #c{background-color:lightsalmon}
            #d{ background-color:lightslategrey }
    </style>
</head>
<body>
  <div id="littlegrid">
    <div id="a">A</div>
    <div id="b">B</div>
    <div id="c">C</div>
    <div id="d">D</div>
  </div>
```



Grid Items: Posicionamento

- As propriedades ***grid-row-gap/grid-column-gap*** permitem estabelecer um espaço entre tracks
  - ***grid-gap*** permite agregar estes dois valores por linha e coluna

```
#littlegrid{
    display: grid;
    grid-auto-flow: column;
    grid-gap: 20px;
    grid-template-columns: 200px 200px;
    grid-template-rows: 200px 200px;
    grid-auto-columns: 100px;
    grid-auto-rows: 100px
}
```



Grid Items: Posicionamento

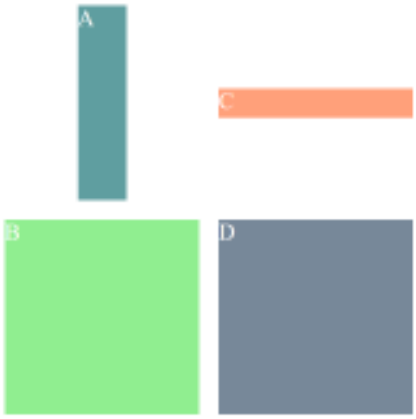
- Quando os items possuem dimensão inferior à grid que foi criada, podem ser alinhados (posicionados) tendo por base as propriedades:
  - ***justify-self*** (alinhamento horizontal)
  - ***align-self*** (alinhamento vertical)

```
#littlegrid{
    display: grid;
    grid-auto-flow: column;
    grid-gap: 20px;
    grid-template-columns: 200px 200px;
    grid-template-rows: 200px 200px;
    grid-auto-columns: 100px;
    grid-auto-rows: 100px    }

#a{background-color: cadetblue;
   width:50px;
   justify-self: center;}

#b{background-color:lightgreen; }

#c{background-color:lightsalmon;
   height:30px;
   align-self:center}
```

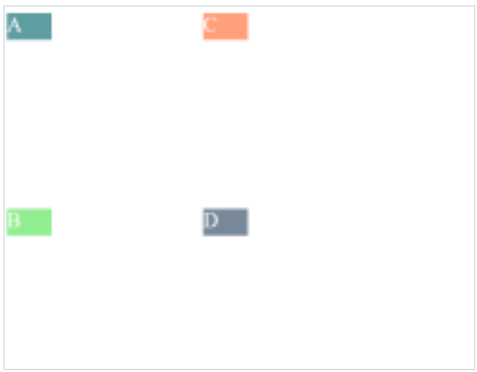


Grid Items: Posicionamento

- A propriedade ***justify-items*** permite o alinhamento horizontal para todos os items

```
div{color:white; font-size: 1.2em;
     width:50px;
     height:30px}

#littlegrid{
    display: grid;
    grid-auto-flow: column;
    justify-items: start;
    grid-gap: 20px;
    grid-template-columns: 200px 200px;
    grid-template-rows: 200px 200px;}
```




```
#littlegrid{
        display: grid;
        grid-auto-flow: column;
        justify-items: end;
        grid-gap: 20px;
        ...
```

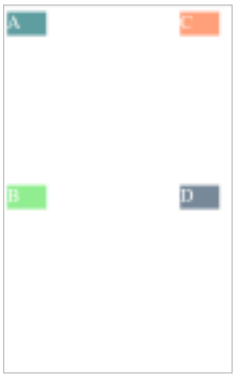


Grid Items: Posicionamento

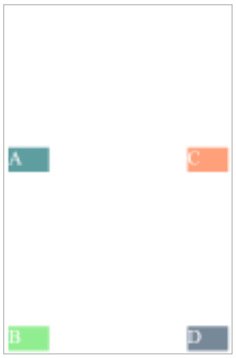
- A propriedade ***align-items*** permite o alinhamento vertical para todos os items

```
div{color:white; font-size: 1.2em;
    width:50px;
    height:30px}

#littlegrid{
    display: grid;
    grid-auto-flow: column;
    align-items: start;
    grid-gap: 20px;
    grid-template-columns: 200px 200px;
    grid-template-rows: 200px 200px}
```




```
#littlegrid{
    display: grid;
    grid-auto-flow: column;
    align-items: end;
    ...
```



CSS Grid / CSS Flexbox





Flexbox – 1D	Mais flexível Menos código Mais fácil de manter	CSS Grid – 2D

Podem ser combinadas!
(ex: disposição de elementos no header/footer)

https://hackernoon.com/the-ultimate-css-battle-grid-vs-flexbox-d40da0449faf

Framework CSS

layouts

Frameworks (layout)

- Bootstrapp
  - Instalação diretamente a partir de um *Content Delivery Network (CDN)*



https://f9official.com/2017/08/28/content-delivery-network-cdn/

  - Download rápido (possibilidade de *pre-cached files*) assim como controlo de versões
    - https://www.sitepoint.com/7-reasons-to-use-a-cdn/

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet>

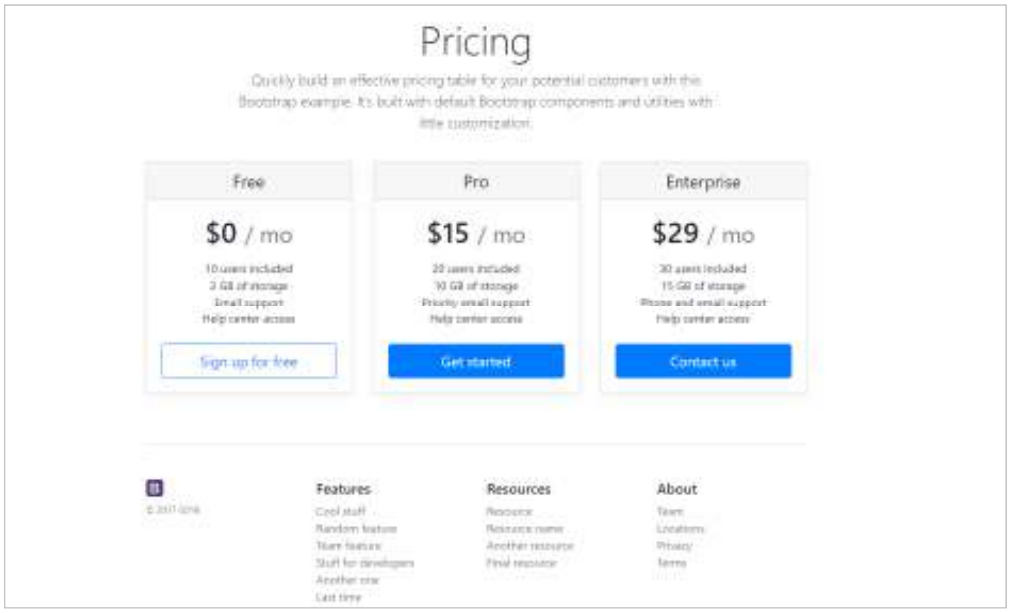
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js" ></script>
```

Frameworks(layout)

- Bootstrapp

All you need to do is learn the class names and apply them to your html markup. They'll do what they are designed to do.

https://medium.freecodecamp.org/bootstrap-4-everything-you-need-to-know-c750991f6784



https://getbootstrap.com/docs/4.0/examples/